

LAPORAN UTS
GRAFIKA KOMPUTER



Nabigh Nailur Ridho

240533608084

(OFFERING C)

UNIVERSITAS NEGERI MALANG
FAKULTAS TEKNIK
PROGRAM STUDI PENDIDIKAN TEKNIK INFORMATIKA
2026

LABORATORIUM KOMPUTER
DTEI FT - Universitas Negeri Malang
Gedung B11 Lantai 2, Malang, Jawa Timur



UTS

Nama	Nabigh Nailur Ridho
Nim	240533608084
Prodi	S1 Pendidikan Teknik Informatika
Dosen Pembimbing	Ir. Azhar Ahmad Smaragdina, M.Pd.
Asisten Praktikum	Tegar Fatur Rachman

UTS

A. Kode Lengkap

```
//
=====
=====
// UTS Grafika Komputer - KODE FINAL (Gabungan Soal 1-5)
// Referensi: LearnOpenGL - Textures (4.2, 4.6) +
Transformations (5.1)
// https://learnopengl.com/Getting-started/Textures
//
https://learnopengl.com/Getting-started/Transformations
//
=====
=====
// NIM: 240533608084 | Mix Value: 0.4 (digit akhir = 4)
//
=====
=====
// Soal 1: Persegi (2D quad) + 2 Texture (Foto Profil +
NIM)
// Soal 2: Mix value 0.4 berdasarkan NIM
// Soal 3: Transformasi berulang (looping): rotasi,
scale, translasi
// Soal 4: Interaksi keyboard & mouse
// Soal 5: Variasi (ubah mix value, ubah transformasi,
toggle animasi)
//
=====
=====
// KONTROL:
// WASD      : Gerak objek (translasi)
// R / T     : Rotasi kiri / kanan
// + / -     : Scale up / down
// Mouse Scroll: Zoom (scale)
// UP / DOWN : Ubah mix value (+/- 0.05)
// 0-9       : Set mix value langsung (0.0 - 0.9)
// SPACE     : Toggle animasi on/off
// C         : Cycle transform mode
(ALL/ROTATE/SCALE/TRANSLATE)
// X         : Reset semua ke default
// ESC       : Keluar
//
=====
=====
```

```

#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <stb_image.h>

#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

#include <learnopengl/shader.h>

#include <iostream>
#include <cmath>
#include <string>
#include <cstdlib>

void framebuffer_size_callback(GLFWwindow* window, int
width, int height);
void processInput(GLFWwindow *window);
void scroll_callback(GLFWwindow* window, double xoffset,
double yoffset);
void key_callback(GLFWwindow* window, int key, int
scancode, int action, int mods);

// settings
const unsigned int SCR_WIDTH = 800;
const unsigned int SCR_HEIGHT = 600;

//
=====
====
// Soal 2: Mix value berdasarkan NIM
// NIM: 240533608084 -> digit akhir = 4 -> mixValue =
0.4
// Referensi: LearnOpenGL - Textures Exercise 4
// stores how much we're seeing of either texture
//
=====
====
float mixValue = 0.4f;

//
=====
====
// Soal 4 & 5: State untuk interaksi dan variasi
//
=====
====
float posX = 0.0f, posY = 0.0f;
float rotation = 0.0f;

```

```

float scaleVal = 0.5f;
bool animationOn = true;
int transformMode = 0;

int main()
{
    // glfw: initialize and configure
    // -----
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE,
GLFW_OPENGL_CORE_PROFILE);

#ifdef __APPLE__
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
#endif

    // glfw window creation
    // -----
    GLFWwindow* window = glfwCreateWindow(SCR_WIDTH,
SCR_HEIGHT,
        "UTS Grafkom - Gabungan Soal 1-5 (NIM:
240533608084)", NULL, NULL);
    if (window == NULL)
    {
        std::cerr << "Failed to create GLFW window" <<
std::endl;
        glfwTerminate();
        system("pause");
        return -1;
    }
    glfwMakeContextCurrent(window);
    glfwSetFramebufferSizeCallback(window,
framebuffer_size_callback);
    glfwSetScrollCallback(window, scroll_callback);
    glfwSetKeyCallback(window, key_callback);

    // glad: load all OpenGL function pointers
    // -----
    if
(!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
    {
        std::cerr << "Failed to initialize GLAD" <<
std::endl;
        system("pause");
        return -1;
    }

    // build and compile our shader program

```

```

// -----
// Referensi: LearnOpenGL Shader class
(learnopengl/shader.h)
    std::string vsPath = std::string(SHADER_DIR) +
"/uts.vs";
    std::string fsPath = std::string(SHADER_DIR) +
"/uts.fs";
    std::cout << "Loading shaders from: " << vsPath <<
std::endl;
    Shader ourShader(vsPath.c_str(), fsPath.c_str());

//
=====
====
    // Soal 1: Set up vertex data untuk PERSEGI (2D
quad)
    // Referensi: LearnOpenGL - Transformations (5.1)
    //
=====
====
    float vertices[] = {
        // positions          // texture coords
        0.5f,  0.5f, 0.0f,    1.0f, 1.0f, // top right
        0.5f, -0.5f, 0.0f,   1.0f, 0.0f, // bottom
right
        -0.5f, -0.5f, 0.0f,   0.0f, 0.0f, // bottom left
        -0.5f,  0.5f, 0.0f,   0.0f, 1.0f  // top left
    };
    unsigned int indices[] = {
        0, 1, 3, // first triangle
        1, 2, 3  // second triangle
    };
    unsigned int VBO, VAO, EBO;
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);

    glBindVertexArray(VAO);

    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices),
vertices, GL_STATIC_DRAW);

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER,
sizeof(indices), indices, GL_STATIC_DRAW);

    // position attribute
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 *
sizeof(float), (void*)0);

```

```

        glEnableVertexAttribArray(0);
        // texture coord attribute
        glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 *
sizeof(float), (void*)(3 * sizeof(float)));
        glEnableVertexAttribArray(1);

        //
=====
=====
        // Soal 1: Load and create textures (2 texture)
        // 1) Foto Profil masing-masing
        // 2) NIM masing-masing
        // Referensi: LearnOpenGL - Textures
(4.2.textures_combined)
        // https://learnopengl.com/Getting-started/Textures
        //
=====
=====
        unsigned int texture1, texture2;
        int width, height, nrChannels;
        stbi_set_flip_vertically_on_load(true);
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1); // Fix untuk
resolusi gambar aneh (2631x2631)

        // Construct texture paths using TEXTURE_DIR defined
by CMake
        std::string texPath1 = std::string(TEXTURE_DIR) +
"/Profile for uts grafkom.jpeg";
        std::string texPath2 = std::string(TEXTURE_DIR) +
"/persegi nim.png";

        // texture 1 - Foto Profil
        // -----
        glGenTextures(1, &texture1);
        glBindTexture(GL_TEXTURE_2D, texture1);
        // set the texture wrapping parameters
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_REPEAT);
        // set texture filtering parameters
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MAG_FILTER, GL_LINEAR);
        // load image, create texture and generate mipmaps
        std::cout << "Loading texture 1: " << texPath1 <<
std::endl;
        unsigned char *data = stbi_load(texPath1.c_str(),
&width, &height, &nrChannels, 0);

```

```

        if (data)
        {
            glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width,
height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
            glGenerateMipmap(GL_TEXTURE_2D);
            std::cout << "    -> OK! " << width << "x" <<
height << " (" << nrChannels << " channels)" <<
std::endl;
        }
        else
        {
            std::cerr << "    -> FAILED to load texture 1
(Foto Profil)" << std::endl;
        }
        stbi_image_free(data);

        // texture 2 - NIM (240533608084)
        // -----
        glGenTextures(1, &texture2);
        glBindTexture(GL_TEXTURE_2D, texture2);
        // set the texture wrapping parameters
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_REPEAT);
        // set texture filtering parameters
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MAG_FILTER, GL_LINEAR);
        // load image, create texture and generate mipmaps
        std::cout << "Loading texture 2: " << texPath2 <<
std::endl;
        data = stbi_load(texPath2.c_str(), &width, &height,
&nrChannels, 0);
        if (data)
        {
            // Referensi: LearnOpenGL - PNG may have alpha
channel (GL_RGBA)
            GLenum format = (nrChannels == 4) ? GL_RGBA :
GL_RGB;
            glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width,
height, 0, format, GL_UNSIGNED_BYTE, data);
            glGenerateMipmap(GL_TEXTURE_2D);
            std::cout << "    -> OK! " << width << "x" <<
height << " (" << nrChannels << " channels)" <<
std::endl;
        }
        else
        {

```



```

        std::cerr << "  -> FAILED to load texture 2
(NIM)" << std::endl;
    }
    stbi_image_free(data);

    // tell opengl for each sampler to which texture
    unit it belongs to (only has to be done once)
    //
    -----
    -----

    ourShader.use(); // don't forget to activate/use the
    shader before setting uniforms!
    ourShader.setInt("texture1", 0);
    ourShader.setInt("texture2", 1);

    // Print info
    std::cout <<
    "\n=====
====" << std::endl;
    std::cout << "UTS GRAFIKA KOMPUTER - KODE FINAL
(Soal 1-5)" << std::endl;
    std::cout <<
    "=====
=" << std::endl;
    std::cout << "NIM: 240533608084 | Mix Value awal: "
    << mixValue << std::endl;
    std::cout << "Referensi: LearnOpenGL (Textures +
Transformations)" << std::endl;
    std::cout << "
https://learnopengl.com/Getting-started/Textures" <<
    std::endl;
    std::cout << "
https://learnopengl.com/Getting-started/Transformations"
    << std::endl;
    std::cout <<
    "=====
=" << std::endl;
    std::cout << "KONTROL:" << std::endl;
    std::cout << "  WASD          : Gerak objek
(translasi)" << std::endl;
    std::cout << "  R / T          : Rotasi kiri / kanan"
    << std::endl;
    std::cout << "  + / -          : Scale up / down" <<
    std::endl;
    std::cout << "  Scroll          : Zoom (scale)" <<
    std::endl;
    std::cout << "  UP / DOWN       : Ubah mix value (+/-
0.05)" << std::endl;
    std::cout << "  0-9          : Set mix value
langsung" << std::endl;

```

```

        std::cout << "    SPACE          : Toggle animasi on/off"
<< std::endl;
        std::cout << "    C              : Cycle transform mode"
<< std::endl;
        std::cout << "    X              : Reset semua" <<
std::endl;
        std::cout << "    ESC          : Keluar" << std::endl;
        std::cout <<
"=====
=\n" << std::endl;

// render loop
// -----
// Referensi: LearnOpenGL - render loop pattern
while (!glfwWindowShouldClose(window))
{
    // input
    // -----
    processInput(window);

    // render
    // -----
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // bind textures on corresponding texture units
    // Referensi: LearnOpenGL -
4.2.textures_combined
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, texture1);
    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, texture2);

    //
=====
====
        // Soal 2 & 5: set the texture mix value in the
shader
        // Referensi: LearnOpenGL - Textures Exercise 4
        //
https://learnopengl.com/Getting-started/Textures
        //
=====
====
        ourShader.setFloat("mixValue", mixValue);

        //
=====
====
        // Soal 3, 4, 5: create transformations

```

```

        // Referensi: LearnOpenGL - Transformations
(5.1)
        //
https://learnopengl.com/Getting-started/Transformations
        //
=====
=====

        float time = (float)glfwGetTime();
        glm::mat4 transform = glm::mat4(1.0f); // make
sure to initialize matrix to identity matrix first

        // Soal 4: Manual translation from keyboard
input
        transform = glm::translate(transform,
glm::vec3(posX, posY, 0.0f));

        // Soal 3 & 5: Animated transformations
(looping)
        if (animationOn)
        {
            // Translasi melingkar (circular
translation)
            if (transformMode == 0 || transformMode ==
3) {
                float radius = 0.2f;
                transform = glm::translate(transform,
glm::vec3(sin(time) * radius, cos(time) * radius,
0.0f));
            }

            // Rotasi terus-menerus (continuous
rotation)
            // Referensi: LearnOpenGL - transform =
glm::rotate(transform, (float)glfwGetTime(), ...)
            if (transformMode == 0 || transformMode ==
1) {
                transform = glm::rotate(transform, time,
glm::vec3(0.0f, 0.0f, 1.0f));
            }

            // Scaling bernapas (breathing scale effect)
            if (transformMode == 0 || transformMode ==
2) {
                float breathScale = 0.5f + 0.15f *
sin(time * 2.0f);
                transform = glm::scale(transform,
glm::vec3(breathScale, breathScale, 1.0f));
            }
        }
    }
}

```

```

        // Soal 4: Manual rotation from keyboard
        transform = glm::rotate(transform,
glm::radians(rotation), glm::vec3(0.0f, 0.0f, 1.0f));

        // Soal 4: Manual scale (selalu diaplikasikan)
        transform = glm::scale(transform,
glm::vec3(scaleVal, scaleVal, 1.0f));

        // get matrix's uniform location and set matrix
        // Referensi: LearnOpenGL - glGetUniformLocation
+ glUniformMatrix4fv
        ourShader.use();
        unsigned int transformLoc =
glmGetUniformLocation(ourShader.ID, "transform");
        glUniformMatrix4fv(transformLoc, 1, GL_FALSE,
glm::value_ptr(transform));

        // render container
        glBindVertexArray(VAO);
        glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT,
0);

        // glfw: swap buffers and poll IO events (keys
pressed/released, mouse moved etc.)
        //
-----
-----

        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    // optional: de-allocate all resources once they've
outlived their purpose:
    //
-----
-----

        glDeleteVertexArrays(1, &VAO);
        glDeleteBuffers(1, &VBO);
        glDeleteBuffers(1, &EBO);

        // glfw: terminate, clearing all previously
allocated GLFW resources.
        //
-----
-----

        glfwTerminate();
        return 0;
    }

    //

```

```

=====
====
// Soal 4: process all input (continuous keys)
// Referensi: LearnOpenGL - processInput pattern
//
https://learnopengl.com/Getting-started/Transformations
//
=====
====
void processInput(GLFWwindow *window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) ==
GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    // Translasi - WASD
    float speed = 0.01f;
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        posY += speed;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        posY -= speed;
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        posX -= speed;
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
        posX += speed;

    // Rotasi - R (kiri) / T (kanan)
    if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS)
        rotation += 2.0f;
    if (glfwGetKey(window, GLFW_KEY_T) == GLFW_PRESS)
        rotation -= 2.0f;

    // Scale - +/- keys
    if (glfwGetKey(window, GLFW_KEY_EQUAL) ==
GLFW_PRESS)
        scaleVal += 0.01f;
    if (glfwGetKey(window, GLFW_KEY_MINUS) ==
GLFW_PRESS)
    {
        scaleVal -= 0.01f;
        if (scaleVal < 0.05f)
            scaleVal = 0.05f;
    }
}

//
=====
====
// Soal 5: key callback untuk event-based input
(variasi)

```

```

// Referensi: LearnOpenGL - Textures Exercise 4 (UP/DOWN
mix value)
// https://learnopengl.com/Getting-started/Textures
//
=====
====
void key_callback(GLFWwindow* window, int key, int
scancode, int action, int mods)
{
    if (action != GLFW_PRESS) return;

    // Soal 5: UP/DOWN untuk mengubah mix value dari
texture
    // Referensi: LearnOpenGL Exercise 4
    if (key == GLFW_KEY_UP)
    {
        mixValue += 0.05f;
        if (mixValue >= 1.0f)
            mixValue = 1.0f;
        std::cout << "Mix Value: " << mixValue <<
std::endl;
    }
    if (key == GLFW_KEY_DOWN)
    {
        mixValue -= 0.05f;
        if (mixValue <= 0.0f)
            mixValue = 0.0f;
        std::cout << "Mix Value: " << mixValue <<
std::endl;
    }

    // Soal 5: Set mix value langsung via angka 0-9
    if (key >= GLFW_KEY_0 && key <= GLFW_KEY_9)
    {
        mixValue = (key - GLFW_KEY_0) * 0.1f;
        std::cout << "Mix Value: " << mixValue <<
std::endl;
    }

    // Soal 5: Toggle animasi on/off
    if (key == GLFW_KEY_SPACE)
    {
        animationOn = !animationOn;
        std::cout << "Animasi: " << (animationOn ? "ON"
: "OFF") << std::endl;
    }

    // Soal 5: Cycle transform mode
    if (key == GLFW_KEY_C)
    {

```

```

        transformMode = (transformMode + 1) % 4;
        const char* modes[] = {"ALL", "ROTATE ONLY",
"SCALE ONLY", "TRANSLATE ONLY"};
        std::cout << "Transform Mode: " <<
modes[transformMode] << std::endl;
    }

    // Soal 5: Reset semua ke default
    if (key == GLFW_KEY_X)
    {
        posX = 0.0f;
        posY = 0.0f;
        rotation = 0.0f;
        scaleVal = 0.5f;
        mixValue = 0.4f;
        animationOn = true;
        transformMode = 0;
        std::cout << "RESET! Mix Value: " << mixValue <<
std::endl;
    }
}

//
=====
====
// Soal 4: scroll callback untuk zoom (scale)
// Referensi: LearnOpenGL - scroll_callback
//
=====
====
void scroll_callback(GLFWwindow* window, double xoffset,
double yoffset)
{
    scaleVal += (float)yoffset * 0.05f;
    if (scaleVal < 0.05f)
        scaleVal = 0.05f;
    if (scaleVal > 3.0f)
        scaleVal = 3.0f;
}

// glfw: whenever the window size changed (by OS or user
resize) this callback function executes
//
-----
-----
void framebuffer_size_callback(GLFWwindow* window, int
width, int height)
{
    // make sure the viewport matches the new window
dimensions; note that width and

```

```
// height will be significantly larger than
specified on retina displays.
glViewport(0, 0, width, height);
}
```

B. Kode dan Penjelasan

1. Membuat Persegi dan 2 Texture

Source code

```
float vertices[] = {
0.5f, 0.5f, 0.0f, 1.0f, 1.0f,
0.5f, -0.5f, 0.0f, 1.0f, 0.0f,
-0.5f, -0.5f, 0.0f, 0.0f, 0.0f,
-0.5f, 0.5f, 0.0f, 0.0f, 1.0f
};

unsigned int indices[] = {
0, 1, 3,
1, 2, 3
};
```

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
5*sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE,
5*sizeof(float), (void*)(3*sizeof(float)));
glEnableVertexAttribArray(1);
```

```
unsigned int texture1, texture2;
glGenTextures(1, &texture1);
glGenTextures(1, &texture2);
```


2. Mix Value Texture

Source code

```
float mixValue = 0.4f;
```

```
ourShader.setFloat("mixValue", mixValue);
```

Penjelasan:

Pada bagian ini, digunakan variabel `mixValue` untuk mengatur perbandingan antara dua tekstur yang ditampilkan pada objek. Nilai awal ditentukan sebesar 0.4, yang berasal dari digit terakhir NIM. Nilai ini menunjukkan seberapa besar kontribusi masing-masing tekstur dalam proses pencampuran. Ketika nilai mendekati 0, tekstur pertama akan lebih dominan, sedangkan jika mendekati 1, tekstur kedua akan lebih terlihat. Nilai ini dikirim ke shader untuk diproses lebih lanjut sehingga menghasilkan efek blending antara dua gambar secara halus.

3. Transformasi Otomatis (Animasi Loop)

Source code

Kode Penting

```
float time = (float)glfwGetTime();  
glm::mat4 transform = glm::mat4(1.0f);
```

Translasi melingkar

```
transform = glm::translate(transform,  
    glm::vec3(sin(time)*0.2f, cos(time)*0.2f, 0.0f));
```

Rotasi terus

```
transform = glm::rotate(transform, time,
    glm::vec3(0.0f, 0.0f, 1.0f));
```

Scaling animasi (breathing)

```
float breathScale = 0.5f + 0.15f * sin(time * 2.0f);
transform = glm::scale(transform,
    glm::vec3(breathScale, breathScale, 1.0f));
```

Penjelasan:

Program menerapkan transformasi secara otomatis menggunakan waktu sebagai parameter utama melalui fungsi `glfwGetTime()`. Transformasi yang dilakukan meliputi translasi melingkar menggunakan fungsi sinus dan cosinus, rotasi kontinu terhadap sumbu z, serta scaling dinamis yang menghasilkan efek membesar dan mengecil seperti bernapas. Semua transformasi ini digabungkan dalam satu matriks transformasi sehingga objek terlihat bergerak secara dinamis tanpa interaksi pengguna. Pendekatan ini menunjukkan pemanfaatan konsep transformasi dalam animasi berbasis waktu.

4. Interaksi Keyboard & Mouse

Source code

Translasi (WASD)

```
if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
    posY += 0.01f;
```

Rotasi manual

```
if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS)
    rotation += 2.0f;
```

Scaling manual

```
scaleVal += 0.01f;
```

Scroll zoom

```
scaleVal += (float)yoffset * 0.05f;
```

Penjelasan:

Pada bagian ini, program memungkinkan pengguna untuk berinteraksi langsung dengan objek melalui input keyboard dan mouse. Tombol WASD digunakan untuk menggeser posisi objek (translasi), tombol R dan T untuk mengatur rotasi, serta tombol + dan - untuk mengubah skala objek. Selain itu, scroll mouse juga digunakan untuk memperbesar atau memperkecil objek secara lebih fleksibel. Dengan adanya fitur ini, pengguna dapat mengontrol transformasi objek secara real-time, sehingga meningkatkan interaktivitas dalam aplikasi grafika.

5. Membuat Persegi dan 2 Texture

Source code

Ubah mix value

```
if (key == GLFW_KEY_UP) mixValue += 0.05f;
```

Set angka langsung

```
mixValue = (key - GLFW_KEY_0) * 0.1f;
```

Toggle animasi

```
animationOn = !animationOn;
```

Mode transform

```
transformMode = (transformMode + 1) % 4;
```

Reset

```
posX = posY = rotation = 0.0f;  
scaleVal = 0.5f;  
mixValue = 0.4f;
```

Penjelasan:

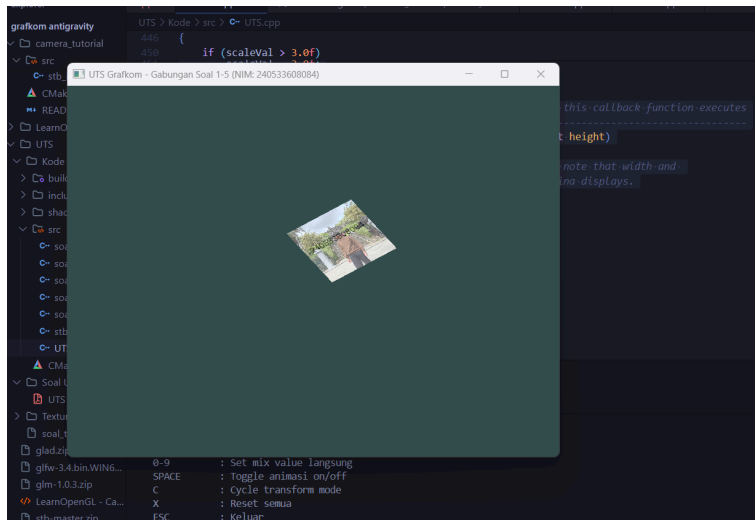
Sebagai pengembangan lebih lanjut, program dilengkapi dengan berbagai fitur tambahan yang meningkatkan fleksibilitas dan interaktivitas. Pengguna dapat mengubah nilai `mixValue` menggunakan tombol panah atau langsung melalui angka 0–9. Selain itu, terdapat fitur untuk mengaktifkan atau menonaktifkan animasi dengan tombol spasi, serta mengubah mode transformasi menggunakan tombol C untuk memilih antara rotasi, scaling, translasi, atau kombinasi semuanya. Tersedia juga fitur reset dengan tombol X untuk mengembalikan semua parameter ke kondisi awal. Variasi ini membuat program menjadi lebih dinamis dan memberikan kontrol yang lebih luas kepada pengguna..

Kesimpulan:

Program ini berhasil mengintegrasikan berbagai konsep dasar dalam grafika komputer menggunakan OpenGL, mulai dari pembuatan objek dua dimensi berbentuk persegi, penerapan dua tekstur, hingga pengaturan pencampuran tekstur menggunakan nilai tertentu. Selain itu, program juga mengimplementasikan transformasi objek seperti translasi, rotasi, dan scaling baik secara otomatis berbasis waktu maupun melalui interaksi pengguna menggunakan keyboard dan mouse. Ditambah dengan fitur variasi

seperti pengaturan mix value, toggle animasi, dan mode transformasi, aplikasi ini menjadi lebih interaktif dan dinamis. Secara keseluruhan, program telah memenuhi seluruh kebutuhan soal UTS serta menunjukkan pemahaman yang baik terhadap konsep rendering, transformasi, dan interaktivitas dalam grafika komputer.

Output:



Link Video:

 **UTS Grafkom.mp4**

C. LINK DRIVE UTS

 **UTS Grafkom**

D. DAFTAR PUSTAKA

LearnOpenGL.

[LearnOpenGL](https://learnopengl.com/)